

AWS EC2 Auto Start-Stop Automation

Project Overview

This project demonstrates how to automate the start and stop operations of Amazon EC2 instances using AWS serverless services. The solution eliminates the need for manual instance management by scheduling EC2 instances to start and stop automatically at predefined times, helping optimize cloud resource utilization and reduce operational costs.

The automation is built using **AWS Lambda (Python 3.14)** and **Amazon EventBridge Scheduler**. Lambda executes Python code to identify EC2 instances based on tags and performs the required start or stop action. EventBridge Scheduler triggers the Lambda function according to the configured schedule, while IAM provides secure permissions for the automation to interact with EC2 resources.

Key Features

- Automatically starts and stops EC2 instances on a schedule.
- Uses **AWS Lambda** with **Python 3.14** for serverless automation.
- Uses **Amazon EventBridge Scheduler** for scheduled execution.
- Uses **IAM Roles and Policies** to securely manage permissions.
- Dynamically identifies EC2 instances using resource tags.
- Eliminates the need to hardcode EC2 instance IDs.
- Supports easy scaling by managing multiple tagged instances.
- Helps reduce AWS costs by running instances only when required.

AWS Services Used

- Amazon EC2
- AWS Lambda
- Amazon EventBridge Scheduler
- AWS Identity and Access Management (IAM)
- Amazon CloudWatch

Project Workflow

1. Create an Amazon EC2 instance and assign the required tag (`AutoStartStop=true`).
2. Create an IAM role with permissions to describe, start, and stop EC2 instances.
3. Develop a Lambda function using Python 3.14 to automate EC2 operations.
4. Configure EventBridge Scheduler to trigger the Lambda function at specific times.
5. Pass the required action (`start` or `stop`) as input to the Lambda function.
6. Lambda identifies the tagged EC2 instance and performs the requested operation.
7. CloudWatch logs capture execution details for monitoring and troubleshooting.

Benefits

- Reduces manual effort through automation.
- Optimizes AWS infrastructure costs.
- Demonstrates serverless and event-driven architecture.
- Improves understanding of IAM permissions and Lambda integrations.
- Provides a scalable solution that can manage multiple EC2 instances using tags.

Technologies Used

- AWS Lambda
- Python 3.14
- Amazon EC2
- Amazon EventBridge Scheduler
- AWS IAM
- Amazon CloudWatch

Learning Outcomes

Through this project, I gained hands-on experience with AWS serverless computing, Lambda function development, IAM role configuration, EventBridge scheduling, CloudWatch monitoring, and automated EC2 lifecycle management. This project strengthened my understanding of event-driven cloud architectures and practical infrastructure automation on AWS.